

Desarrollo de Software 2026

Desarrollo Frontend Moderno: Introducción a React

Clase #08 - 02 de junio del 2026

TEMAS

- **Comprender qué son los frameworks frontend**
- **Conocer React y sus ventajas**
- **Configurar un entorno de desarrollo**
- **Crear nuestros primeros componentes**



Problemas con Vanilla JavaScript

- Manipulación compleja del DOM
- Código repetitivo y difícil de mantener
- Difícil organización en proyectos grandes
- Reutilización limitada de código

```
// Por ejemplo: Para mostrar una lista de usuarios...
document.getElementById('user-list').innerHTML = '';
users.forEach(user => {
  const div = document.createElement('div');
  div.className = 'user-card';
  div.innerHTML = `

### ${user.name}</h3><p>${user.email}</p>`; document.getElementById('user-list').appendChild(div); });


```

HERRAMIENTAS FRONTEND MODERNAS

Conjunto de librerías y frameworks que facilitan el desarrollo de aplicaciones web

¿Qué nos dan?



Estructura organizada del código



Reutilización de componentes



Mejor rendimiento



Herramientas de desarrollo



Comunidad y documentación

Ejemplos populares:

React (Biblioteca) | Vue.js | Angular (framework)

¿POR QUÉ REACT?

Creado por Facebook (Meta) en 2013

Usado por: Netflix, Airbnb, Instagram, WhatsApp Web, Mercado Libre



Ventajas clave

- Componentes reutilizables
- Virtual DOM = mejor rendimiento
- Curva de aprendizaje suave
- Comunidad masiva
- Alta demanda laboral

En Argentina aproximadamente el 70% de las ofertas frontend requieren React



4 CONCEPTOS CLAVE DE REACT

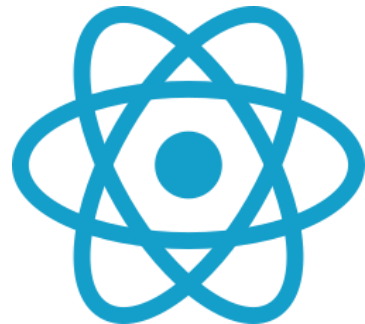
- 1 JSX
- 2 COMPONENTES
- 3 PROPS
- 4 ESTADO



Dominando estos 4 conceptos, ya pueden crear aplicaciones increíbles



Vamos uno por uno...





JSX - JS + HTML

¿Qué es JSX?

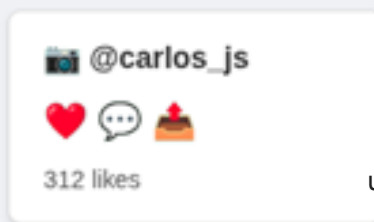
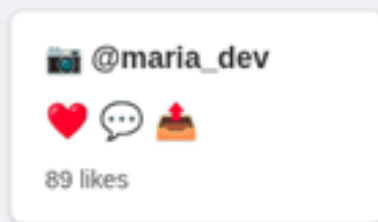
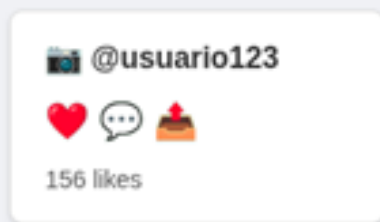
JavaScript que parece HTML, pero NO es HTML

```
jsx
function TarjetaProducto() {
  const producto = "Smartphone";
  const precio = 899;
  const disponible = true;
  const descuento = precio * 0.1;
  return (
    <div className="producto">
      <h2>{producto}</h2>           {/* Variable JS */}
      <p>${precio}</p>              {/* Número JS */}
      <p>Descuento: ${descuento}</p>  {/* Operación JS */}
      <p>Stock: {disponible ? "✅" : "❌"}</p>  {/* Condición JS */}
    </div>
  );
}
```

COMPONENTES

¿Qué son?

Piezas reutilizables de interfaz de usuario



↑ Mismo
componente
"PostCard"

usado 3 veces

```
jsx
function PostCard(props) {
  return (
    <div className="post">
      <h3>📷 {props.usuario}</h3>
      <div>❤️ 💬 🍰</div>
      <p>{props.likes} likes</p>
    </div>
  );
}
```

 1 componente



∞ usos con datos diferentes

PROPS

¿Qué son?

Argumentos que se le pasan a un componente

```
jsx
function PeliculaCard(props) {
  return (
    <div className="movie-card">
      <img src={props.imagen} />
      <h3>{props.titulo}</h3>
      <p>★ {props.rating}/5</p>
      <p>{props.año} • {props.genero}</p>
    </div>
  );
}
```

```
// Un componente, tres películas diferentes
<PeliculaCard titulo="Avengers" rating={5} año={2019} genero="Acción" />
<PeliculaCard titulo="Stranger Things" rating={5} año={2022} genero="Sci-fi" />
<PeliculaCard titulo="Narcos" rating={4} año={2015} genero="Drama" />
```

 1 molde + diferentes datos



infinitas variaciones



ESTADO

¿Qué es?

Información que puede cambiar y hace que el componente se actualice

```
jsx
// ANTES (sin estado): botón "muerto"
<button>❤️ Like</button>  {/* Siempre igual */}

// DESPUÉS (con estado): botón "vivo"
function BotonLike() {
  const [likes, setLikes] = useState(0);
  const liked = likes > 0;
  return (
    <button onClick={() => {
      setLikes(likes + 1)
    }}>
      {liked ? "❤️ " : "💜 "} {likes} likes
    </button>
  );
}
```

Click → Estado cambia →
Pantalla se actualiza
¡AUTOMÁTICAMENTE!



Estado = memoria del componente



Props vienen de afuera

COMPARACIÓN

ANTES

javascript

```
let count = 0;
document.getElementById('button').addEventListener('click', function() {
  count++;
  document.getElementById('counter').innerHTML = count;
});
```

DESPUÉS

jsx

```
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(count + 1)}>
      {count}
    </button>
  );
}
```



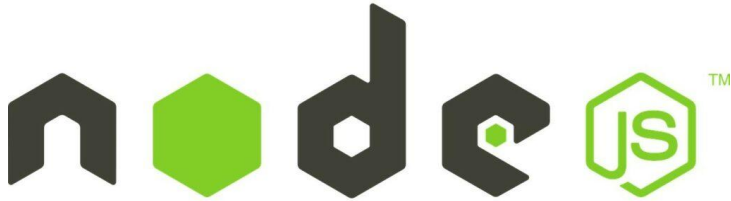
Resultado:
¡El mismo comportamiento!



Diferencia:
Menos código, más claro



Instalación de Entorno



Vite



PREPARANDO NUESTRO ENTORNO

Necesitamos:

- 1 Node.js (runtime de JavaScript)
- 2 NPM (gestor de paquetes)
- 3 Vite (herramienta oficial recomendada)

VITE v6.3.5 ready in 341 ms

→ Local: <http://localhost:5173/>
→ Network: use --host to expose
→ press h + enter to show help

[React.dev: Crear App](#)

```
bash
```

```
### Parado en el directorio donde irán los proyectos
```

```
# Crear la App con Vite:
```

```
npm create vite@latest mi-primera-app -- --template react
```

```
cd mi-primera-app # Ingresar al directorio del proyecto
```

```
npm install        # Instalar las dependencias
```

```
npm run dev        # Iniciar el servidor local
```

```
# Acceder con el navegador a http://localhost:5173/
```



ESTRUCTURA DEL PROYECTO

```
mi-primer-app/  
├── public/           ← Archivos estáticos  
├── src/              ← Nuestro código  
│   ├── App.jsx      ← Componente principal  
│   ├── main.jsx     ← Punto de entrada  
│   └── App.css       ← Estilos  
├── index.html        ← Index HTML donde se inyecta la App React  
├── package.json      ← Dependencias configuradas  
└── node_modules/    ← Código fuente de dependencias instaladas
```



Archivos importantes

App.jsx: Aquí trabajaremos principalmente

main.jsx: Conecta React con el DOM



CREANDO NUESTRO PRIMER COMPONENTE

```
react

// Componente simple
function MiPrimerComponente() {
  return <h1>¡Hola desde React!</h1>;
}

// Usando el componente
function App() {
  return (
    <div>
      <MiPrimerComponente />
      <MiPrimerComponente />
      <MiPrimerComponente />
    </div>
  );
}
```



Puntos clave:

Los componentes son funciones que retornan JSX

Nombres siempre en PascalCase

Se usan como etiquetas HTML: <MiComponente />



PROPS: PASANDO DATOS A COMPONENTES

```
react

// Componente que recibe props
function TarjetaPersona(props) {
  return (
    <div className="tarjeta">
      <h2>{props.nombre}</h2>
      <p>Edad: {props.edad}</p>
      <p>Profesión: {props.profesion}</p>
    </div>
  );
}

// Usando el componente con diferentes datos
function App() {
  return (
    <div>
      <TarjetaPersona
        nombre="Ana"
        edad={28}
        profesion="Desarrolladora"
      />
      <TarjetaPersona
        nombre="Carlos"
        edad={35}
        profesion="Diseñador"
      />
    </div>
  );
}
```

Recursos y Referencias

Documentación Oficial

- [React Dev](#)
- Será la fuente principal de seguimiento

Setup Recomendado

- ⚡ Vite (npm create vite@latest)
-  React Developer Tools (extensión Chrome)
-  VS Code Extensions: ES7+ React/Redux/React-Native snippets